Multimedia Tools and Applications (2023) 82:27973–27987
https://doi.org/10.1007/s11042-023-14572-1

Audio block encryption using 3D chaotic system with adaptive parameter perturbation

Hongjun Liu [1]

Received: 7 March 2021 / Revised: 20 July 2022 / Accepted: 31 January 2023 /
Published online: 13 March 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract
To ensure the sensitive audio data transmit securely over the insecure internet, we proposed a chaos based dual-channel audio block encryption algorithm. A 3D chaotic system is served as pseudo random number generator to encrypt the audio data in time domain through confusion and diffusion. The encryption process includes, (1) generating initial values of chaotic system based on random external values and hash value of plain audio; (2) before encrypting each block, iterate the chaotic system with adaptive parameter perturbation to generate confusion and diffusion sequences; (3) divide the plain audio data into blocks with the same size, confuse and diffuse the time domain vector of each block. Experimental and analysis results demonstrated that, for a 60 second dual-channel wave file with 44.1 kHz and 16-bit quantization, its encryption or decryption speed is only about 1 second, hence the proposed scheme is so suitable for real-time secure communication.

Keywords Audio block encryption · Time domain · Lorenz system · Impulsive perturbation

1 Introduction

With the fast development of modern wireless communication and multimedia technologies, a huge amount of sensitive audio data needs to be transmitted over the insecure internet [7], however, without encrypted audio or speech data over instant messenger is easily intercepted, hence, to design an fast scheme to encrypt & decrypt short speech piece is very necessary for instant messenger. Generally, the length of speech over instant messenger is less than one minute, hence, it is necessary to design a short speech piece encryption algorithm.

There are several kinds of encryption algorithms for audio signal, such as time domain, amplitude and time domain scrambling, 2D mixed scrambling method that combines frequency and time domain [16]. Due to good pseudo-randomness and sensitivity to initial conditions,

✉ Hongjun Liu
sms_liuhj@ujn.edu.cn

[1] School of Mathematical Sciences, University of Jinan, Jinan 250022, China

the original or optimized chaotic system can be applied to design audio encryption algorithm. Parvees et al. [20] designed a chaos-based audio encryption algorithm using confusion and diffusion techniques. Li et al. [10] designed an audio encryption using synchronization between two hyper-chaotic systems. Kordov [9] designed an audio encryption algorithm with chaotic circle map and modified rotation equations. Naskar et al. [17] designed an audio encryption method based on DNA encoding and channel shuffling. Renza et al. [24] proposed a speech encryption method using a variable-length encoding technique based on the Collatz conjecture. Omar et al. [18] used El-Gamal algorithm to encrypt speech signal, the results confirmed the robustness against noise effect. Samuel et al. [25] designed a new secure speech communication scheme based on hyperchaotic masking and modulation. Dai et al. [5] designed an audio encryption algorithm based on Chen memristor chaotic system.

Hardware implementation of audio encryption scheme further enhanced their practical application. Chang et al. [3] proposed a digital programmable multi-scroll chaotic system and applied to FPGA-based audio secure communication, used feedback drive-response synchronization. Abdulaziz et al. [1] studied the dynamic behavior of chaotic system at three different orders, employed them in a sound encryption scheme and achieved good results. Ahmed et al. [2] introduced design and FPGA implementation of sound encryption system based on a fractional-order chaotic system. Tolba et al. [27] proposed a chaotic oscillator with odd/even nonlinearity and FPGA implementation, applied to speech encryption and achieved a throughput exceeding 1Gbit/sec. Piotr et al. [21] proposed a novel distortion-tolerant speech encryption scheme for secure voice communication. Gan et al. [6] designed an ARM-embedded implementation multicast scheme for multiuser speech wireless communication.

Impulsive perturbation to state variable or control parameter, is an effective way to change the trajectory of a chaotic system. Ren et al. [23] dealt with the fixed time stability of delayed complex dynamical networks system with impulsive pinning controllers, which can save more energy and reduce control costs. He et al. [8] investigated the stability of nonlinear variable-time impulsive differential systems with delayed impulses. Ren et al. [22] analyzed a random impulsive switched time-delay system with asynchronous impulses and switches. Liu et al. [13] designed an image encryption scheme using complex hyper chaotic system by injecting impulse into parameters.

In the field of audio encryption, the scramble algorithm in time domain based on FFT can guarantee the cipher of audio remains the features of audio signal. Chen et al. [4] designed a speech encryption algorithm that scrambled the speech signal in time domain based on FFT transformation domain, only the spectral position within 300HZ-3400HZ were replaced, so the bandwidth of the speech signal has not been extended. Liu et al. [12] proposed a whole audio stream encryption scheme by confusion and diffusion based on multi-scroll chaotic system and one-time keys.

Now days, plain speech communication through instant messenger can lead to privacy leakage, hence, secure communication is urgently needed. However, instant messenger with secure communication requires real-time audio block encryption and decryption, and generally the maximize speech duration is limited to 60-second.

To solve the problem of audio secure transmission over insecure internet, we try to design a novel dual-channel audio block encryption scheme based on a 3D chaotic system, (1) generating initial values of chaotic system based on random external values and hash value of plain audio; (2) before encrypting each block, iterate the chaotic system with adaptive parameter perturbation to generate confusion and diffusion sequences; (3) divide the plain audio data into blocks with the same size, confuse and diffuse the time domain vector of each

block. Audio data with more channels can also be encrypted using this method. Security analysis will be used to demonstrate the sensitivity to keys, and experimental statistical results are used to verify the effectiveness of the proposed audio encryption scheme.

The remainder is organized as follows: Section 2 briefly introduces the dynamic characteristics of Lorenz system. Section 3 proposes the audio encryption and decryption algorithms. Section 4 is experimental results. Section 5 is security and performance analysis. Section 6 reaches a conclusion.

2 The Lorenz system

The nonlinear differential equation of Lorenz system [28] can be expressed using Eq. (1).

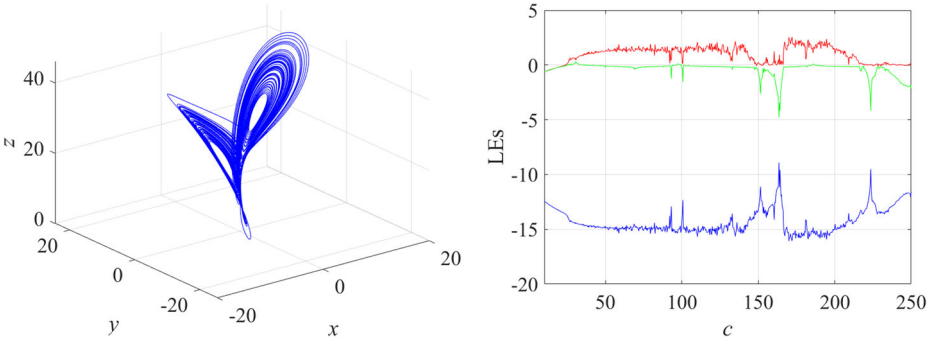
$$\begin{cases} x' = a(y-x) \\ y' = cx-y-xz \\ z' = xy-bz \end{cases}, \tag{1}$$

here $\mathbf{X} = [xyz]^{\mathrm{T}}$ refers to three state variables, two common parameters $a = 10$ and $b = 8/3$, the control parameter $c = 28$ and can vary in a larger chaotic interval. Three equilibrium points include $(0,\ 0,\ 0)$ and $\left(\pm\sqrt{b*(c-1)}, \pm\sqrt{b*(c-1)}, c-1\right)$. The 3D attractor projection shown in Fig. 1a demonstrates that the system can stay in chaotic state, and from Fig. 1b we can infer that the Lorenz system can stay chaotic state in a larger interval of control parameter c.

To investigate the ergodicity of three state time sequences generated by Eq. (1), first we need to iterate Eq. (1) 200 times with random initial values $x(0), y(0), z(0) \in (0,\ 1)$ and default parameter values, to eliminate the transient process, then continue to iterate it 20,000 times to get three double precision floating point number sequences $\{x\}$, $\{y\}$, $\{z\}$, their decimal parts $\{x_d\}$, $\{y_d\}$, $\{z_d\}$ can be obtained using Eq. (2).

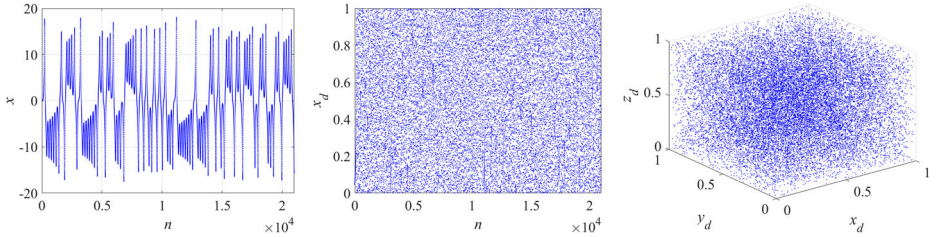
$$\begin{cases} x_d = \mathrm{mod}(x, 1) \\ y_d = \mathrm{mod}(y, 1) \\ z_d = \mathrm{mod}(z, 1) \end{cases}. \tag{2}$$

From the time domain diagrams of $\{x\}$and $\{x_d\}$ shown in Fig. 2a-b, we can infer that the decimal part has ergodicity and better randomness, and from the 3D phase diagram of $\{x_d\}$,



(a) 3D attractor projection (b) Lyapunov exponents

Fig. 1 3D attractor and Lyapunov exponents



(a) Time domain diagram of $\{x\}$ (b) Time domain diagram of $\{x_d\}$ (c) 3D phase diagram of decimal parts

Fig. 2 Time domain diagram and phase diagram

$\{y_d\}$, $\{z_d\}$ shown in Fig. 2c, we can further infer that the decimal parts have ergodicity in the phase space, hence Eq. (1) can be served as a pseudo-random number generator.

3 Audio encryption and decryption algorithms

3.1 Plain audio data

The sound can be recorded by any voice recording equipment, such as mobile phone or recording pen, the sampled audio data may be single-channel, dual-channel, or multi-channel, and dual-channel is the most common used data type format. A wave file can be read and return the data type of double, the normalized matrix element values between [−1.0, 1.0], and the sample rate in Hertz.

3.2 Generating initial values

Here we first generate three initial values of Eq. (1) through combining the internal initial key with the external key. The internal key is a 256-bit hash value H generated from the plain audio A_P, and the external keys are sampled from true random noise, we can combine them to generate three initial values of Eq. (1) using Eq. (3). By this method, two audio data with even only one-bit difference can also generate two different hash values, then further generate three completely different initial values of Eq. (1).

The 256-bit hash value can be expressed as a hexadecimal number array H.

$$H = (h_1, h_2, \ldots, h_{64}).$$

The initial values of $x(0)$, $y(0)$ and $z(0)$ can be obtained by Eq. (3).

$$\begin{cases} x(0) = x'(0) + \mathrm{fix}\left(\mathrm{hex2dec}(H(h_1 : h_{22}))/10^{14}\right) \\ y(0) = y'(0) + \mathrm{fix}\left(\mathrm{hex2dec}(H(h_{23} : h_{44}))/10^{14}\right) \\ z(0) = z'(0) + \mathrm{fix}\left(\mathrm{hex2dec}(H(h_{45} : h_{64}))/10^{14}\right) \end{cases}, \tag{3}$$

where $x'(0)$, $y'(0)$ and $z'(0)$ are external initial values that sampled from true random noise, the function $\mathrm{fix}(x)$ is to round the float number x to the nearest integer towards zero, $\mathrm{hex2dec}(x)$ can convert a hexadecimal number x to its decimal number format.

3.3 Divide dual-channel vector into blocks

Transform the dual-channel audio A_P into a time domain matrix M with the size of L-by-2. To speed up the encryption process, we reshape M to get a $(2L/n_b)$-by-n_b array, where n_b is the block size and can set to be 2^n, $n = 2, 3, 4, \ldots$. It should be noted that if $2L$ cannot be evenly divided by n_b, we can add padding to the end of M, to achieve the expected length. The left half and the right half of n_b can be encrypted by different variable sequences generated by Eq. (1).

3.4 Encryption algorithm

The encryption algorithm includes confusion process and diffusion process, the former is to permute the element positions of two channels, and the latter is to change their values through a nonlinear transformation. The purpose is to transform the plain audio with distinct time domain waveform into a cipher audio with relatively uniform and random distribution, the purpose is to resist against common attacks.

Input: The plain audio A_P with dual-channel, the external initial values: $x'(0)$, $y'(0)$ and $z'(0)$.

Output: The cipher audio A_C.

Step 1. First obtain the hash value H of A_P, then calculate the initial values of Eq. (1) using Eq. (3).

Step 2. Transform A_P into a time domain matrix M with the size of L-by-2, then reshape it into a $(2L/n_b)$-by-n_b array M_R using Eq. (4).

$$M_R = \textbf{\textit{reshape}}(M, \mathbf{2}L/n_b, n_b). \tag{4}$$

Step 3. Solve Eq. (1) by Runge-Kutta method, first iterate it 200 times to eliminate the transient process, then continue to iterate it $2L/n_b$or more times, to obtain three double precision floating-point number sequences $\{x(i)\}$, $\{y(i)\}$, $\{z(i)\}$. Finally, get their decimal parts $\{x_d(i)\}$, $\{y_d(i)\}$, $\{z_d(i)\} \in [0, 1)$ through modulo 1, $i = 1, 2, \ldots, 2L/n_b$.

The adaptive updating rule of parameter sequence $\{c(i)\}$ for the i-th iteration can be defined using Eq. (5).

$$c(i) = c(0) + \sqrt{z(i-1) + 1}, \tag{5}$$

where $c(0) = 28$, and $z(i - 1)$ is generated by the $(i$-1)-th iteration.

Step 4. Generating two permutation sequences: to permute the rows of M_R, we first generate two auxiliary arrays R_1 and R_2 using Eq. (6), which represent the row-number of M_R in an ascending order.

$$\begin{cases} R_1 = (1, 2, \ldots, 2L/n_b) \\ R_2 = (1, 2, \ldots, 2L/n_b) \end{cases}. \tag{6}$$

Then permute R_1 and R_2 using Eq. (7) to obtain $R_1^{'}$ and $R_2^{'}$.

$$\begin{cases} R_1^{'}(i) = R_1(\text{floor}(x_d(i) \times 2L/n_b)\text{mod}i) \\ R_2^{'}(i) = R_2(\text{floor}(z_d(i) \times 2L/n_b)\text{mod}i) \end{cases}, \tag{7}$$

where $i = 2L/n_b, 2L/n_b - 1, \ldots, 2$. By this method we can ensure that the index sequences $R_1^{'}$ and $R_2^{'}$ can be permuted without repetition. It should be noted that if $\boldsymbol{floor}(x_d(i) \times 2L/n_b)modi = = 0$, we need to replace $x_d(i)$ with $x_d(i+1)$, and so on, hence Eq. (1) may be iterated more than $2L/n_b$ times.

Step 5. Confusion process: divide M_R into two arrays, the column of left part is numbered as $[1, 2L/n_b]$, and the right part is numbered as $[2L/n_b + 1, n_b]$, then permute them respectively using Eq. (8) to obtain a vector M_{RP}.

$$\begin{cases} M_{RP}(1:2L/n_b, 1:n_b/2) = M_R\left(R_1^{'}(1:2L/n_b), 1:n_b/2\right) \\ M_{RP}(1:2L/n_b, n_b/2+1:n_b) = M_R\left(R_2^{'}(1:2L/n_b), n_b/2+1:n_b\right) \end{cases}. \tag{8}$$

Step 6. Generating two diffusion sequences $D_1 = (D_1(1), D_1(2), \ldots D_1(2L/n_b))$ and $D_2 = (D_2(1), D_2(2), \ldots D_2(2L/n_b))$ using Eqs. (9) and (10) respectively.

$$D_1(i) = \begin{cases} -\text{mod}(x_d(i), 1), \text{if mod}(i, 2) == 0 \\ \text{mod}(x_d(i), 1), \text{if mod}(i, 2) == 1 \end{cases}, \tag{9}$$

$$D_2(i) = \begin{cases} -\text{mod}(z_d(i)), \text{if mod}(i, 2) == 1 \\ \text{mod}(z_d(i)), \text{if mod}(i, 2) == 0 \end{cases}. \tag{10}$$

Step 7. Diffusion process: a lossy nonlinear transformation [12] expressed using Eq. (11) is applied to diffuse each channel, to further enhance the diffusion intensity.

$$M_{RPC} = \frac{D \times (9 + D \times M_{RP} - M_{RP})}{10}, \tag{11}$$

where $D \in (-1.0, 1.0)$ refers to the key-stream produced by Eqs. (9) and (10), and $M_{RP} \in (-1.0, 1.0)$ represents the audio vector to be diffused. For each channel, the lossy diffusion can be expressed using Eq. (12) in detail.

$$\begin{cases} M_{RPC}(i,1:n_b/2) = \dfrac{D_1(i) \times (9 + D_1(i) \times M_{RP}(i,1:n_b/2) - M_{RP}(i,1:n_b/2))}{10} \\ M_{RPC}(i,n_b/2+1:n_b) = \dfrac{D_2(i) \times (9 + D_2(i) \times M_{RP}(i,n_b/2+1:n_b) - M_{RP}(i,n_b/2+1:n_b))}{10}, \end{cases} \quad (12)$$

where $i = 1, 2, \ldots, 2L/n_b$.

Step 8. Reshape M_{RPC} into a L-by-2 array to obtain the ciphered audio A_C.

The flowchart of the encryption algorithm is shown in Fig. 3.

3.5 Decryption algorithm

The decryption algorithm is an inverse process of the encryption algorithm.

Input: The ciphered audio A_C and a 256-bit hash value H.

Output: The deciphered audio A_P.

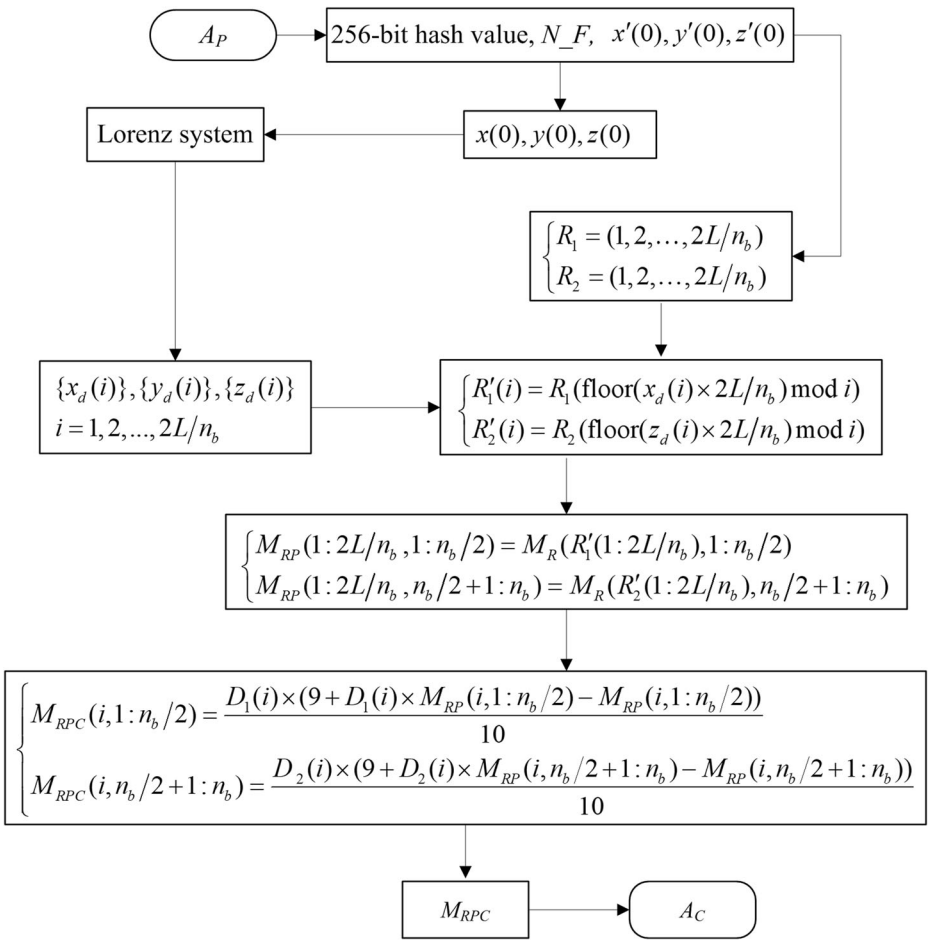


Fig. 3 Flowchart of encryption algorithm

Step 1. Calculate the initial values using Eq. (3).

Step 2. Read the ciphered dual-channel audio A_c, to obtain the time domain matrix M as a L-by-2 vector, then reshape it into a $(2L/n_b)$-by-n_b array M_{RPC} using Eq. (13).

$$M_{RPC} = \textbf{\textit{reshape}}(M, \boldsymbol{2}L/n_b, n_b). \tag{13}$$

Step 3. is same to steps 3–4 of the encryption algorithm, to obtain three double precision floating-point number sequences $\{x(i)\}$, $\{y(i)\}$, $\{z(i)\}$, and two auxiliary arrays $R_1^{'}$ and $R_2^{'}$.

Step 4. is same to step 6 of the encryption algorithm, to generate two diffusion sequences D_1 = ($D_1(1)$, $D_1(2)$, …$D_1(2L/n_b)$) and D_2 = ($D_2(1)$, $D_2(2)$, …$D_2(2L/n_b)$) using Eqs. (9) and (10).

Step 5. Inverse diffusion process: according to Eq. (13), the M_{RP} can be obtained by Eq. (14).

$$M_{RP} = \frac{10M_{RPC} - 9D}{D^2}. \tag{14}$$

The inverse diffusion process for each channel can be expressed using Eq. (15).

$$\begin{cases} M_{RP}(i, 1 : n_b/2) = \dfrac{10 \times M_{RPC}(i, 1 : n_b/2) - 9 \times D_1(i)}{D_1(i) \times D_1(i)} \\ M_{RP}(i, n_b/2 + 1 : n_b) = \dfrac{10 \times M_{RPC}(i, n_b/2 + 1 : n_b) - 9 \times D_2(i)}{D_2(i) \times D_2(i)} \end{cases}. \tag{15}$$

Step 6. Inverse confusion process: inversely confuse the rows of M_{RP}, to obtain a vector M_R using Eq. (16).

$$\begin{cases} M_R\left(R_1^{'}(1 : 2L/n_b), 1 : n_b/2\right) = M_{RP}(1 : 2L/n_b, 1 : n_b/2) \\ M_R\left(R_2^{'}(1 : 2L/n_b), n_b/2 + 1 : n_b\right) = M_{RP}(1 : 2L/n_b, n_b/2 + 1 : n_b) \end{cases}. \tag{16}$$

Step 7. Finally, reshape M_R into a L-by-2 array M using Eq. (17), to obtain the deciphered audio A_P.

$$M = \textbf{\textit{reshape}}(M_R, L, \boldsymbol{2}). \tag{17}$$

The flowchart of the decryption algorithm is shown in Fig. 4.

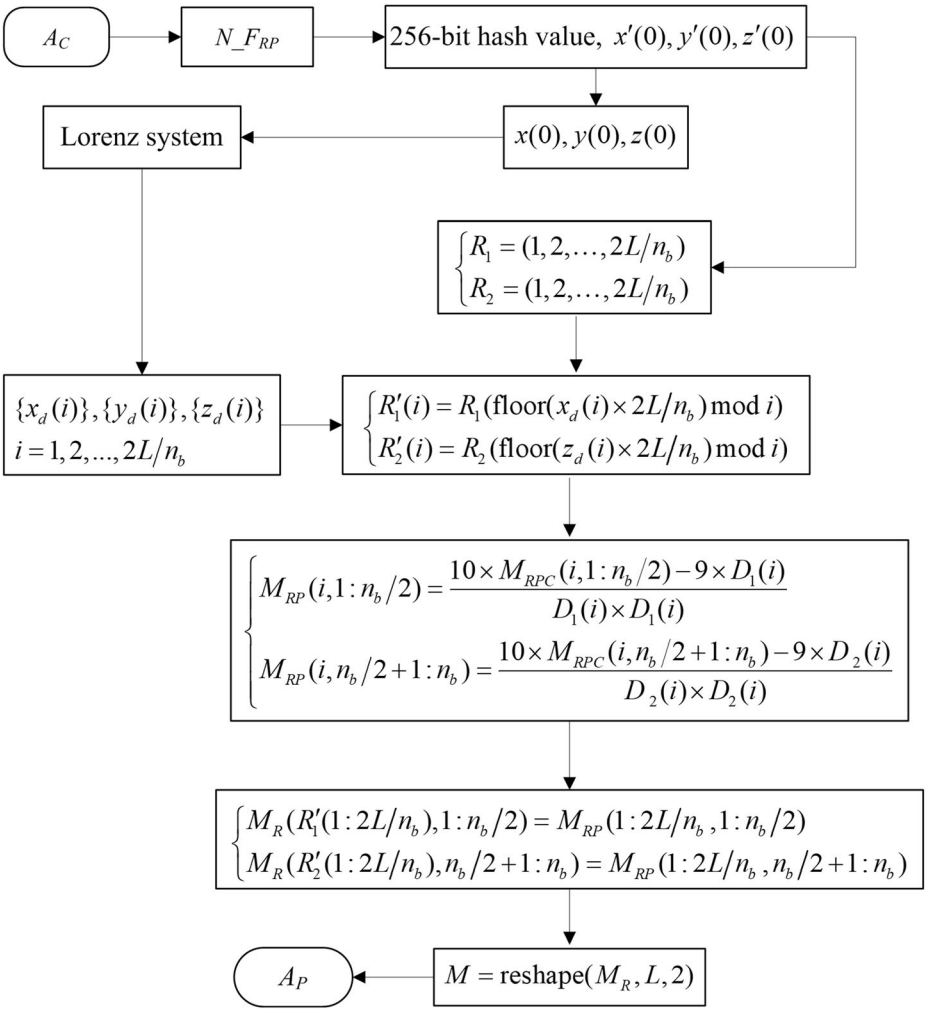


Fig. 4 Flowchart of decryption algorithm

4 Experimental results

The experimental results are based on the following initial conditions: (1) H = "DE9F52186309C71A52630678C09F2BAFF1F7075B2D2264910FCDB7DD78F59D88"; (2) parameters a = 10, b = 8/3 and c = 28; (3) the external initial values $x'(0)$ = 0.792207329559554, $y'(0)$ = 0.959492426392903 and $z'(0)$ = 0.655740699156587.

The time domain diagram of the sample plain audio, the confusion and diffusion results and deciphered audio with $n_b = 2^2$ and $n_b = 2^3$ are shown in Figs. 5 and 6 respectively. The amplitude error between the plain audio and deciphered audio is very tiny, as shown in Fig. 7. From which we can infer that, the cipher audio is completely undistinguished, and the error between the plain audio and deciphered audio is so tiny that they sound exactly the same.

In practical application, the block size, i.e., time duration, can be adjusted and optimized, to achieve a balance between speed and intensity. As a lossy audio encryption, from Fig. 7 we

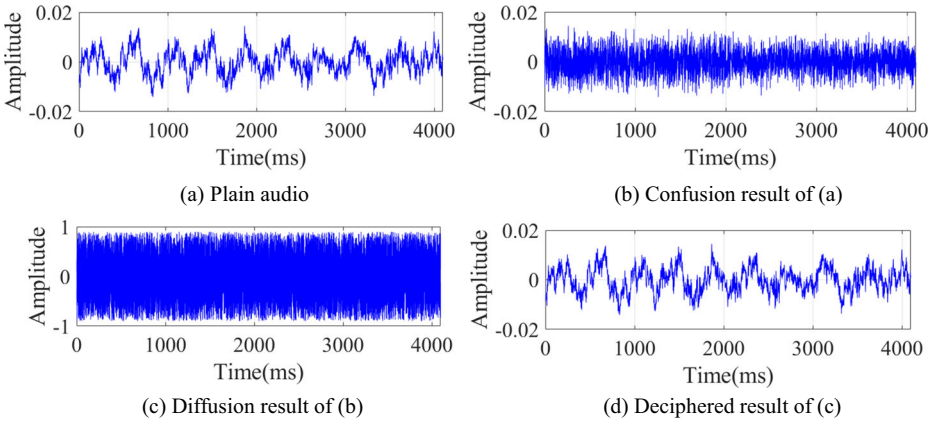


Fig. 5 Time domain diagram ($n_b = 2^2$)

can infer that that the error between the plain audio and the deciphered audio is so tiny that it is imperceptible by ears.

5 Security and performance analysis

Security and performance analysis are conducted to test the experimental results, including key space, key sensitivity, correlation, histogram, peak signal-to-noise ratio, speed and differential attack.

5.1 Key space

As a symmetric encryption scheme, the key space can be calculated as follows: (1) the 256-bit hash value of plain audio, its key space can be $S = 2^{128} \approx 3.4 \times 10^{38}$; (2) the external initial values $x'(0)$, $y'(0)$ and $z'(0)$. Generally, the valid precision of a continuous chaotic system can

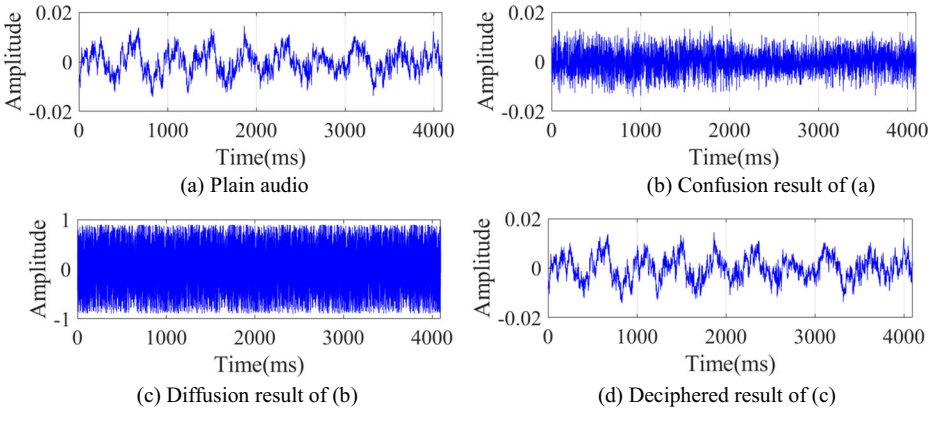
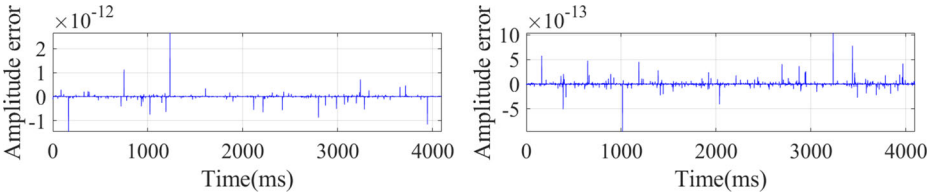


Fig. 6 Time domain diagram ($n_b = 2^3$)



(a) Amplitude error between Fig.5 (d) and Fig. 5(a) (b) Amplitude error between Fig.6 (d) and Fig. 6(a)

Fig. 7 Amplitude error between plain audio and deciphered audio with different n_b

reach 10^{-14} [19], hence their key space can be 10^{42}. Therefore, the total key space $S \approx 3.4 \times 10^{38} \times 10^{42} = 3.4 \times 10^{80}$ [11], which is large enough to resist exhaustive attacks.

5.2 Key sensitivity analysis

High sensitivity to initial values of a chaotic system can bring high security to an encryption scheme. The security of proposed algorithm mainly depends on the external one-time keys transformed from natural noise, even two identical audios can also produce two completely different ciphers. To test the key-sensitivity to the initial values, we add a small value to $x(0)$, to decrypt different ciphers encrypted by different block sizes, the testing results are shown in Table 1, from which we can infer the algorithm is highly dependent on small change of initial value. The security also partly rely on the internal key, which is generated by the hash value of the plain audio, one-bit change to plain audio can lead to about 50% bit-flip.

5.3 Correlation analysis

The correlation r between two audios of vectors A and B, can be expressed using Eq. (18), where $\overline{A}$ and $\overline{B}$ are mean values of A and B.

$$r = \frac{\sum_{i=1}^{n}\left(A[i]-\overline{A}\right)\left(B[i]-\overline{B}\right)}{\sqrt{\sum_{i=1}^{n}\left(A[i]-\overline{A}\right)^2 \sum_{i=1}^{n}\left(B[i]-\overline{B}\right)^2}}. \tag{18}$$

Lower correlation between the plain audio and its cipher means less resemblance and more resistance to attacks. Here we select five samples and test their correlation of each sample's left channel (L) and right channel (R), the subscripts P and C denote the plain audio and the cipher

Table 1 Key-sensitivity testing

Block size	$x(0)+10^{-8}$	$x(0)+10^{-9}$	$x(0)+10^{-10}$	$x(0)+10^{-11}$	$x(0)+10^{-12}$	$x(0)+10^{-13}$
$n_b=2^2$	Undistinguished	Undistinguished	Undistinguished	Barely distin-guished	Distinguished with loud noise	Distinguished with much noise
$n_b=2^3$	Undistinguished	Barely distinguished	Distinguished with loud noise	Distinguished with much noise	Distinguished with little noise	Distinguished with less noise

Table 2 Correlation between two channels with $n_b = 2^2$

Correlation	Plain audio	Cipher audio	Plain audio and cipher	
	L_P/R_P	L_C/R_C	L_P/L_C	R_P/R_C
Sample1	0.999999	−0.008759	−0.004923	−0.008993
Sample2	0.600503	−0.065297	−0.002498	−0.001008
Sample3	0.623679	0.142928	0.001808	0.004719
Sample4	0.722581	0.184132	0.001600	−0.001235
Sample5	0.626737	0.032393	−0.001939	−0.002822
Ref. [20]	N/P	0.0263	N/P	N/P
Ref. [10]	0.9830	0.0057	N/P	N/P
Ref. [5]	0.9921	0.0043	0.0014	0.0021
Ref. [26]	N/P	N/P	0.0312	0.0458
Sample1 [12]	0.999999	0.004312	−0.003194	−0.000358
Sample2 [12]	0.739449	0.002812	0.000650	0.001555

audio, respectively. The results shown in Table 2 demonstrated that, the smaller the n_b, the smaller the correlation.

5.4 Histogram analysis

Histogram analysis can help to examine whether the distribution of cipher is uniform, because it is more difficult to attack a cipher with uniform distribution [14]. Here the histograms of the left and right channels of plain audio Sample1 and its ciphers with different n_b are shown in Fig. 8, which demonstrates that the distribution of the cipher audio becomes more uniform.

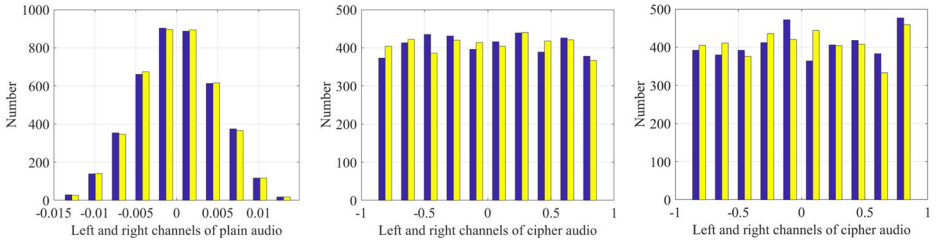
5.5 Peak signal-to-noise ratio (PSNR)

For any plain audio A_p and its cipher A_c, the mean squared error can be calculated using Eq. (19).

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(A_p[i]-A_c[i]\right)^2, \tag{19}$$

and the PSNR can be computed using Eq. (20).

$$\boldsymbol{PSNR} = \boldsymbol{10log_{10}}\left(\frac{(\boldsymbol{2^n-1})^2}{\mathrm{MSE}}\right). \tag{20}$$



(a) Plain audio (b) Cipher audio with $n_b=2^2$ (c) Cipher audio with $n_b=2^3$

Fig. 8 Histograms of plaintext and cipher

Table 3 Results of PSNR with different block sizes

Audios	PSNR (n_b=2^2)	PSNR (n_b=2^3)
Sample1	−1.7632	−1.9217
Sample2	2.1027	2.1672
Sample3	2.9595	2.9533
Sample4	5.5062	5.3325
Sample5	−6.9798	−6.7725
Ref. [20]	4.373	N/P
Ref. [5]	4.6302	4.4186
Sample1 [12]	4.5299	N/P
Sample2 [12]	4.4733	N/P

Table 3 shows the PSNR of all the ciphers, a lower PSNR indicates a higher level of noise in the cipher audio, which means more resistance to attacks.

5.6 Speed analysis

Due to the iterations of Eq. (1) is dependent on the block size, the speed comparison with different n_b is shown in Table 4. For a 60 second dual-channel wave file with 44.1 kHz sample frequency, 16-bit quantization, its size can be 60*44100*16*2/1024/1024 ≈ 80.7495Mbit, the encryption or decryption speed is about 1 second, hence the speed is suitable for real-time secure communication. The equipment of PC is: Windows 10, 64-bit, Matlab R2018b, Intel Core i7-6500U CPU @2.50GHz, RAM 16.00GB memory.

5.7 Differential attack analysis

The differential attack analysis [15] is applied to verify whether the cipher is sensitive to tiny change of the plaintext, i.e., satisfies the avalanche effect, which can be measured by NPCR (number of pixels change rate) and UACI (unified average changing intensity). For a tiny change to a plain audio can lead to a hash value with about 50% bit flipping. Here we verified the effect through changing 1-bit to the plain audio, and then combined the hash values with external initial values into initial values of Lorenz system, to ensure the proposed encryption scheme can resist differential attack. From Table 5, we can infer that our results are all close to the ideal values of 99.6094070% and 33.4635070%.

Table 4 Encryption speed comparison with different n_b

n_b	2^2	2^3	2^4	2^5	2^6	2^7
Speed (Mbps)	80.6452	92.5926	89.2857	92.8677	88.8652	85.1498

Table 5 NPCR and UACI with comparison

	NPCR(%)	UACI(%)
Ref. [5]	99.9269	32.8195
Ref. [6]	99.61	33.47
Ref. [22]	99.6132	33.4519
Ref. [12]	99.6019	33.2339
Our scheme	99.5937	33.4656

6 Conclusion

To ensure the audio data transmit over insecure network securely, a novel chaos-based dual-channel block audio encryption algorithm is proposed. A 3D chaotic system with impulsive perturbation is used to confuse and diffuse the audio's time domain. The main novelty includes: (1) iterating the chaotic system with an adaptive parameter perturbation to resist common attacks, and (2) diffusing and confusing each audio block with matrix and modular operation can achieve a high speed. The proposed scheme has the characteristic of flexible block size, fast encrypting & decrypting algorithm, and sensitivity to tiny change of plain audio. Analysis results demonstrated that the scheme can be applied to multi-channel real-time block encryption over instant messenger. In the future, we will further study the practical application of chaos-based real-time audio encryption to instant messenger.

Acknowledgments This research is supported by the Natural Science Foundation of Shandong Province (No. ZR2022MF232), the General Research Project of Liaoning Provincial Education Department of China (No. LJKZ1185).

Declarations I declare that we have no conflict of interest.

Data sharing not applicable to this article as no datasets were generated or analyzed during the current study.

References

1. Abdulaziz H, Mohammed F, Lobna A et al (2020) Enhanced hardware implementation of a mixed-order nonlinear chaotic system and speech encryption application. AEU Int J Electron Commun 125:153347
2. Ahmed J, El-Maksoud A et al (2019) FPGA implementation of sound encryption system based on fractional-order chaotic systems. Microelectron J 90:323–335
3. Chang D, Li Z, Wang M et al (2018) A novel digital programmable multi-scroll chaotic system and its application in FPGA-based audio secure communication[J]. AEU-Int J Electr Commun 88:20–29
4. Chen Y, Hao J, Chen J et al (2015) End-to-end speech encryption algorithm based on speech scrambling in time domain[C]. //Third International Conference on Cyberspace Technology (CCT 2015), pp 1–5. https://doi.org/10.1049/cp.2015.0820
5. Dai W, Xu X, Song X, Li G (2022) Audio encryption algorithm based on Chen memristor chaotic system[J]. Symmetry 14(1):17
6. Gan Q, Yu S, Li C, Lü J, Lin Z, Chen P (2016) Design and ARM-embedded implementation of a chaotic map-based multicast scheme for multiuser speech wireless communication[J]. Int J Circuit Theory Appl 45(11):1849–1872
7. Ghasemzadeh H, Khasss MT, Mehrara H (2017) Cipher text only attack on speech time scrambling systems using correction of audio spectrogram[J]. ISeCure 9(2):33–47
8. He Z, Li C, Cao Z, Li H (2021) Stability of nonlinear variable-time impulsive differential systems with delayed impulses[J]. Nonl Anal Hybrid Syst 39:100970

9. Kordov K (2019) A novel audio encryption algorithm with permutation-substitution architecture[J]. Electronics 8(5):530
10. Li G, Pu Y, Yang B, Zhao J (2019) Synchronization between different hyper chaotic systems and dimensions of cellular neural network and its design in audio encryption[J]. Clust Comput 22:7423–7434
11. Liu H, Kadir A (2015) Asymmetric color image encryption scheme using 2D discrete-time map[J]. Signal Process 113:104–112
12. Liu H, Kadir A, Li Y (2016) Audio encryption scheme by confusion and diffusion based on multi-scroll chaotic system and one-time keys[J]. Optik-Int J Light Electron Opt 127(19):7431–7438
13. Liu H, Zhang Y, Kadir A, Xu Y (2019) Image encryption using complex hyper chaotic system by injecting impulse into parameters[J]. Appl Math Comput 360:83–93
14. Liu H, Kadir A, Liu J (2019) Color pathological image encryption algorithm using arithmetic over Galois field and coupled hyper chaotic system[J]. Opt Lasers Eng 122:123–133
15. Liu H, Liu J, Ma C (2022) Constructing dynamic strong S-box using 3D chaotic map and application to image encryption. Multimed Tools Appl https://doi.org/10.1007/s11042-022-12069-x.
16. Mahdi A, Jawad AK, Hreshee S (2016) Digital chaotic scrambling of audio based on duffing map[J]. Communications 1(2):16–21
17. Naskar PK, Paul S, Nandy D, Chaudhuri A (2019) DNA encoding and channel shuffling for secured encryption of audio data[J]. Multimed Tools Appl 78:25019–25042
18. Omar A, Sura F et al (2020) Implementation of El-Gamal algorithm for speech signals encryption and decryption. Proced Comput Sci 167:1028–1037
19. Öztürk İ, Kılıç R (2015) A novel method for producing pseudo random numbers from differential equation-based chaotic systems[J]. Nonlinear Dyn 80(3):1147–1157
20. Parvees MYM, Samath JA, Bose BP (2018) Audio encryption-a chaos-based data byte scrambling technique[J]. Int J Appl Syst Stud 8(1):51–75
21. Piotr K, Jerome L, Bruno M (2022) A novel distortion-tolerant speech encryption scheme for secure voice communication. https://doi.org/10.1016/j.specom.2022.06.007
22. Ren W, Xiong J (2019) Stability analysis for random impulsive switched time-delay systems with asynchronous impulses and switches[J]. Syst Control Lett 133:104516
23. Ren H, Shi P, Deng F, Peng Y (2020) Fixed-time synchronization of delayed complex dynamical systems with random perturbation via impulsive pinning control[J]. J Franklin Inst 357(17):12308–12325
24. Renza D, Mendoza S, Ballesteros DM (2019) High-uncertainty audio signal encryption based on the Collatz conjecture[J]. J Inf Secur Appl 46:62–69
25. Samuel A, Binoy K (2022) A new secure speech communication scheme based on Hyperchaotic masking and modulation. IFAC Papers Line 55(1):914–919
26. Sathiyamurthi P, Ramakrishnan S (2020) Speech encryption algorithm using FFT and 3D-Lorenz-logistic chaotic map[J]. Multimed Tools Appl 79(3):17817–17835
27. Tolba M, Elwakil A et al (2020) FPGA implementation of a chaotic oscillator with odd/even symmetry and its application. Integration 72:163–170
28. Wang X, Wang M (2008) A hyperchaos generated from Lorenz system [J]. Physica A: Stat Mech Appl 387(14):3751–3758

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.